

Submit VII

Algoritmos de Búsqueda y Ordenamiento

Juan Diego Collazos Mejia
Juan Sebastian Garizao Puerto

Shearsort

Análisis Asintótico

El algoritmo **Shearsort** ordena una matriz $n \times n$ en forma serpenteante (“snake-like”), alternando el orden de las filas y ordenando las columnas en cada iteración. Cada ronda consiste en:

1. Ordenar todas las filas (en direcciones alternadas).
2. Ordenar todas las columnas en orden ascendente.

Estas dos fases se repiten $\lceil \log_2 n \rceil + 1$ veces.

Versión Secuencial

 `shearsort-seq.cpp` in GitHub

Cada paso de ordenamiento (fila o columna) tiene una complejidad de $O(n \log n)$ y se aplica a n filas o columnas. Por lo tanto, el costo total del algoritmo secuencial es:

$$T_{seq}(n) = (\log_2 n + 1) \times [O(n \cdot n \log n)] = O(n^2(\log n)^2)$$

Versión Paralela

 `shearsort-par.cpp` in GitHub

En la versión paralela del algoritmo Shearsort, la matriz se divide lógicamente de modo que cada hilo (o proceso) pueda trabajar de manera independiente sobre una porción de los datos. En particular, cada hilo puede ordenar simultáneamente una o varias filas o columnas, lo que permite que las operaciones de ordenamiento que antes eran secuenciales se ejecuten en paralelo.

Dado que Shearsort realiza múltiples fases alternadas de ordenamiento de filas y columnas, el paralelismo se aplica principalmente en dos niveles:

- **Paralelismo de datos:** cada hilo se encarga de un subconjunto de filas o columnas, reduciendo el tiempo total de ordenamiento.
- **Paralelismo de fases:** mientras algunos hilos finalizan su tarea en una etapa, otros pueden comenzar la siguiente, aprovechando la arquitectura multinúcleo.

El tiempo total de ejecución puede modelarse como:

$$T_{par}(n, p) = O\left(\frac{n^2(\log n)^2}{p}\right) + O(\log p)$$

donde el primer término representa la ganancia obtenida al dividir el trabajo entre los p hilos disponibles, y el segundo término $O(\log p)$ modela la sobrecarga debida a la sincronización, intercambio de datos y coordinación entre los hilos (por ejemplo, al finalizar cada fase de ordenamiento y antes de la siguiente transposición).

El *speedup* teórico se define como la razón entre el tiempo secuencial y el paralelo:

$$S_p = \frac{T_{seq}(n)}{T_{par}(n, p)} \approx p \quad (\text{idealmente})$$

En la práctica, el *speedup* real es menor debido a las dependencias entre fases y la sobrecarga de comunicación. No obstante, para tamaños de entrada grandes, el costo del procesamiento domina sobre la sincronización, y el algoritmo puede alcanzar un desempeño casi lineal con el número de hilos.

Tablas de Resultados

Shearsort Secuencial

Cuadro 1: Rendimiento del algoritmo Shearsort en versión secuencial.

Tamaño matriz ($n \times n$)	Lado (n)	Walltime (s)
5×5	5	$1,563 \times 10^{-5}$
10×10	10	$6,9291 \times 10^{-5}$
20×20	20	$3,48395 \times 10^{-4}$
40×40	40	$1,76384 \times 10^{-3}$
80×80	80	$7,87223 \times 10^{-3}$
160×160	160	$3,76065 \times 10^{-2}$
320×320	320	$1,62058 \times 10^{-1}$
640×640	640	$7,12204 \times 10^{-1}$
1280×1280	1280	3,3122
2560×2560	2560	16,051

Shearsort Paralelo

Cuadro 2: Rendimiento del algoritmo Shearsort en versión paralela (8 hilos).

Tamaño matriz ($n \times n$)	Lado (n)	Walltime (s)	Speedup
5×5	5	$1,313020 \times 10^{-3}$	0.012
10×10	10	$2,576080 \times 10^{-3}$	0.027
20×20	20	$3,333640 \times 10^{-3}$	0.105
40×40	40	$5,998830 \times 10^{-3}$	0.294
80×80	80	$9,138590 \times 10^{-3}$	0.861
160×160	160	$1,707240 \times 10^{-2}$	2.203
320×320	320	$5,841160 \times 10^{-2}$	2.774
640×640	640	$1,88219 \times 10^{-1}$	3.784
1280×1280	1280	$8,11136 \times 10^{-1}$	4.083
2560×2560	2560	4,18438	3.836

Análisis de Resultados

Los resultados experimentales obtenidos confirman de manera general el comportamiento teórico esperado para el algoritmo **Shearsort**. En la versión secuencial, el tiempo de ejecución aumenta de forma acelerada a medida que crece el tamaño de la matriz. Dado que la complejidad asintótica es $O(n^2(\log n)^2)$, el crecimiento observado es cuadrático con un leve factor logarítmico adicional, lo que se aprecia en los incrementos progresivos del tiempo cuando n se multiplica por dos o cuatro. Esto refleja el costo acumulado de ordenar todas las filas y columnas repetidamente a lo largo de $\lceil \log_2 n \rceil + 1$ fases.

Para las matrices pequeñas ($n \leq 80$), los tiempos son del orden de milisegundos, mientras que para tamaños grandes ($n \geq 1280$), los tiempos se elevan hasta decenas de segundos, evidenciando el carácter computacionalmente intensivo del algoritmo.

En cuanto a la versión paralela, se observa que el desempeño no mejora de forma inmediata en los tamaños reducidos. De hecho, para matrices pequeñas ($n < 100$) el tiempo paralelo supera al secuencial, debido a que la sobrecarga de creación, sincronización y comunicación entre los hilos es mayor que el beneficio de la división del trabajo. Este fenómeno es característico en algoritmos paralelos de granularidad fina, donde el número de operaciones por hilo no compensa el costo de coordinación.

Sin embargo, conforme el tamaño del problema aumenta, el paralelismo empieza a ofrecer ganancias significativas. A partir de $n = 160$, el speedup comienza a superar la unidad, alcanzando valores cercanos a 4 con 8 hilos cuando $n \geq 1280$. Esto indica que el algoritmo se comporta de manera escalable para tamaños de entrada grandes, aproximándose parcialmente al speedup ideal teórico $S_p \approx p$.

El rendimiento se estabiliza en los casos más grandes ($n = 2560$), lo que sugiere que la eficiencia se ve limitada por el ancho de banda de memoria y por las fases secuenciales inherentes al algoritmo, de acuerdo con la Ley de Amdahl. En resumen:

- El crecimiento temporal en la versión secuencial coincide con la complejidad $O(n^2(\log n)^2)$.
- La versión paralela presenta una sobrecarga inicial elevada, pero obtiene mejoras notorias a partir de tamaños medianos.
- El speedup se aproxima al comportamiento esperado teórico, aunque con saturación en valores entre 3.5 y 4 debido a limitaciones de comunicación y sincronización.

Estos resultados demuestran que Shearsort es un algoritmo inherentemente paralelo, pero su eficiencia depende fuertemente del tamaño del problema y de la proporción entre el trabajo útil y la sobrecarga de coordinación.

Binary Search

Análisis Asintótico

El algoritmo de búsqueda binaria (**Binary Search**) busca un elemento x en un arreglo ordenado dividiendo el espacio de búsqueda a la mitad en cada iteración.

Versión Secuencial

 `bsa-seq.cpp` in GitHub

En la versión secuencial, el algoritmo compara en cada paso el elemento central del arreglo con el valor buscado. Cada iteración reduce el espacio de búsqueda a la mitad, por lo que el tiempo total es:

$$T_{seq}(n) = O(\log n)$$

Versión Paralela (P-BSA)

 `bsa-par.cpp` in GitHub

La versión paralela del algoritmo de búsqueda binaria (**P-BSA**) busca reducir el tiempo de búsqueda dividiendo el conjunto de datos en p subsecuencias, una para cada procesador o hilo. De este modo, cada unidad de procesamiento trabaja sobre un fragmento distinto del arreglo de entrada, verificando si el valor buscado x podría encontrarse dentro de su rango.

En cada paso del algoritmo:

- El arreglo se divide equitativamente en p segmentos.
- Cada procesador examina los valores en los extremos de su subsecuencia (inicio y fin) para determinar si x podría estar contenido en ese rango.
- Si un procesador encuentra que x se encuentra en su rango, notifica a los demás, y el proceso se repite recursivamente, subdividiendo ese rango en p nuevas subsecuencias más pequeñas.

A diferencia de la búsqueda binaria secuencial, donde en cada iteración solo se examina una posición central, en la versión paralela se inspeccionan simultáneamente varios rangos, lo que permite acelerar la localización de la región donde se encuentra el elemento buscado.

En el caso ideal, si la carga de trabajo está perfectamente balanceada y no existe sobrecarga de comunicación, el tiempo de ejecución disminuiría proporcionalmente al número de procesadores:

$$T_{par}(n, p) = O\left(\frac{\log n}{p}\right)$$

Sin embargo, en sistemas reales siempre existe un costo adicional asociado a la sincronización, coordinación y comunicación entre procesadores. Por ello, el modelo más realista incluye un término adicional de complejidad logarítmica:

$$T_{par}(n, p) = O\left(\frac{\log n}{p} + \log p\right)$$

El término $O(\log p)$ representa la sobrecarga de intercambio de mensajes o de control entre los hilos durante la subdivisión del arreglo y la propagación de resultados.

El *speedup* teórico, que mide la ganancia de rendimiento con respecto a la versión secuencial, se expresa como:

$$S_p = \frac{T_{seq}(n)}{T_{par}(n, p)} \approx \frac{p}{1 + \frac{p \log p}{\log n}}$$

Este modelo muestra que el speedup crece con p , pero de forma sublineal debido a las pérdidas producidas por la comunicación. En la práctica, el paralelismo resulta beneficioso únicamente cuando el tamaño del arreglo n es suficientemente grande, de modo que el trabajo computacional $O(\log n)$ domina frente a la sobrecarga de coordinación.

Tablas de Resultados

Binary Search Secuencial

Cuadro 3: Rendimiento del algoritmo de búsqueda binaria secuencial (BSA).

Tamaño (n)	Walltime (s)
10^1	$2,9 \times 10^{-7}$
10^2	$2,8 \times 10^{-7}$
10^3	$5,7 \times 10^{-7}$
10^4	$4,3 \times 10^{-7}$
10^5	$9,8 \times 10^{-7}$
10^6	$1,21 \times 10^{-6}$
10^7	$3,79 \times 10^{-6}$
10^8	$3,72 \times 10^{-6}$
109	$6,09 \times 10^{-6}$

Parallel Binary Search (P-BSA)

Cuadro 4: Rendimiento del algoritmo de búsqueda binaria paralela (P-BSA).

Tamaño (n)	Walltime (s)	Speedup
10^1	$3,140650 \times 10^{-4}$	0.000923
10^2	$1,461420 \times 10^{-4}$	0.001916
10^3	$1,541330 \times 10^{-4}$	0.003698
10^4	$7,762630 \times 10^{-4}$	0.000554
10^5	$8,029120 \times 10^{-4}$	0.001221
10^6	$1,162070 \times 10^{-3}$	0.001041
10^7	$7,092110 \times 10^{-4}$	0.005344
10^8	$6,858610 \times 10^{-4}$	0.005424

Análisis de Resultados

Los resultados obtenidos para el algoritmo de **búsqueda binaria** reflejan con claridad la diferencia entre el comportamiento teórico ideal y el rendimiento observado en la práctica. En la versión secuencial, el tiempo de ejecución crece muy lentamente con el tamaño del arreglo, lo que es consistente con su complejidad $O(\log n)$. Incluso para valores de n del orden de 10^8 , los tiempos permanecen en el rango de microsegundos, mostrando la gran eficiencia del método secuencial. Este comportamiento se debe a que el número de comparaciones aumenta de manera logarítmica, y el costo de acceso a memoria domina sobre el costo computacional.

En contraste, los resultados de la versión paralela (**P-BSA**) evidencian que el tiempo de ejecución es considerablemente mayor que el de la versión secuencial para todos los

tamaños de entrada. El *speedup* medido fue menor que 1 en todos los casos, lo cual indica que la sobrecarga introducida por la creación de hilos, la división del arreglo, la comunicación entre ellos y la sincronización final supera el tiempo que la búsqueda secuencial tarda en completarse.

Este fenómeno es esperable desde el punto de vista teórico. Dado que el costo secuencial es $O(\log n)$, el trabajo computacional total es muy pequeño, y por tanto la paralelización no logra compensar el costo adicional de coordinación. En términos prácticos, la versión paralela solo podría superar a la secuencial si se aplicara sobre volúmenes de datos extremadamente grandes o si se ejecutaran múltiples búsquedas simultáneamente en el mismo conjunto de datos (es decir, paralelismo a nivel de tareas).

A pesar de que el modelo teórico sugiere un tiempo ideal $T_{par}(n, p) = O(\frac{\log n}{p})$, los resultados reales muestran que el término de sobrecarga $O(\log p)$ y las latencias de memoria dominan completamente la ejecución. Así, el rendimiento del P-BSA está limitado por la baja granularidad del problema y por la ley de Amdahl, que establece que las partes no paralelizables imponen un límite máximo al speedup alcanzable.

En resumen:

- La versión secuencial presenta un comportamiento estable y extremadamente eficiente, cumpliendo con la complejidad $O(\log n)$.
- La versión paralela obtiene tiempos mayores debido a la escasa cantidad de trabajo por hilo y a los costos de comunicación y sincronización.
- El speedup observado ($S_p < 1$) confirma que la búsqueda binaria no se beneficia de la paralelización directa sobre una sola consulta, siendo más apropiado aplicar paralelismo a un nivel superior (por ejemplo, múltiples búsquedas en paralelo o procesamiento por lotes).

Por lo tanto, el análisis experimental concuerda con lo esperado teóricamente: la paralelización de un algoritmo con costo logarítmico no genera beneficios cuando el problema de base no posee suficiente carga computacional para amortizar el costo del paralelismo.